

AI Systems Performance Engineering

Chapter 5 — GPU-Based Storage I/O Optimizations

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

September 2026

Roadmap: Five Parts, Each Answers One Question

Part	The question it answers
I Storage fundamentals	How do we lay out and read data so disks deliver peak bandwidth?
II Direct GPU data paths	How does data skip the CPU bounce buffer on its way to HBM?
III Shared storage at scale	What serves many nodes without becoming the bottleneck?
IV The data pipeline	How do loaders and preprocessing keep every GPU fed?
V Profiling workflow	Is the job I/O-, communication-, or compute-bound ?

The whole chapter in one sentence

Keep the data pipeline as full as the compute pipeline — a starving GPU is a wasted GPU.

When a detail page appears (`io_uring`, `cuFile`, `3FS`, `DALI...`), it always belongs to one of these five questions.

Part I — Feeding Data Is as Important as the Compute Itself

- Picture a **100-trillion-parameter** model training on thousands of GPUs over **billions of samples** — terabytes of text, petabytes of images.
- If the storage pipeline is slow, **GPUs starve and sit idle** — low utilization despite every network optimization from Chapter 4.
- This chapter: read data efficiently from disk or remote storage, preprocess it, and **overlap its I/O with GPU compute**.^[1]

Plain English: the world's finest kitchen plates nothing if the pantry line is empty — feeding the factory is a job of its own.

Sizing the appetite (book example)

Each GPU needs **200 MB/s** of training data:

8 GPUs	~ 1.6 GB/s
GB200 NVL72 (72 GPUs)	14–20 GB/s

Heavier multimodal samples? Calibrate with **measured** bytes-per-sample × samples-per-second — demand can be much higher.

Data Locality: Put the Data Physically Close to the Compute

- “Close” = **local NVMe SSD** on the node, or at least **in-rack** via NVMe-over-Fabrics (**NVMe-oF**) — fewer network hops, more consistent performance.
- Network storage (NFS, Amazon S3)? Verify the **aggregate read bandwidth** across all compute nodes covers the appetite from the last page.
- Parallel filesystems (Lustre, GPFS) can **cache data on local SSDs**.

Plain English: don't make every workshop order from the overseas warehouse — stock each workshop's own shelf with the parts it will use.

Shard the dataset across nodes

100 TB, 10 nodes $\Rightarrow$ **presplit 10 TB** onto each node's local disk; each node's loader reads **only its local shard** — no redundant network reads.

- PyTorch's **DistributedSampler** coordinates workers so each process gets a **unique slice** per epoch — aligns perfectly with per-node sharding.

Sequential Beats Random: Shard Your Samples

- Storage delivers far higher throughput for **large sequential reads** than for small random ones — and GPUs want data in **large contiguous chunks**.
- Anti-pattern: **millions of individual image files** ⇒ random seeks all over the disk.
- Instead: pack samples into a few **large binary shards** — Arrow, TFRecord, Parquet, WebDataset tar — one read returns **many samples**.
- Same on object stores: **combine small S3 objects** into larger ones ahead of time.

Plain English: pick up a full pallet per warehouse trip, not one screw at a time — and pre-pack the screws into pallets before training starts.

Tune the read size too

Reading in **1 MB** chunks beats **4 KB** chunks — lower per-read overhead. Loader libraries expose **buffer** and **prefetch chunk** sizes; tune them even though the libraries are internally optimized.

- Book warning: with today's faster GPUs, small random reads become the bottleneck **sooner** — verify your filesystem's small-read behavior explicitly.

When Reads Must Be Random: Parallelism and the Right Filesystem

Hide latency with many in-flight reads

Issue **multiple reads in parallel**: threads calling `pread()`, or Linux async I/O via `io_uring` — preregistered buffers + polling submit **batches of I/O** with minimal kernel overhead $\Rightarrow$ high IOPS.

- Buffered I/O and big read-ahead help **sequential** scans; random reads benefit little — **batch** them or use a dataset API (TFRecordDataset, IterableDataset + DataLoader).

*Plain English: if you must fetch single screws, send **many runners at once** and stop stamping a timecard on every trip.*

Filesystem choices (book tips)

- **XFS** is common on Linux NVMe servers; mount with `noatime` — no access-time write on every read.
- Amazon EFS: use **Max I/O** performance mode; switch Bursting $\rightarrow$ **Provisioned throughput** for consistent bandwidth.

NVMe and Kernel Tuning: Scheduler, Read-Ahead, RAM

- Modern Linux: multiqueue block I/O (**blk-mq**) spreads I/O across cores. For NVMe use the **none** or **mq-deadline** scheduler (CFQ is obsolete); verify via `/sys/block/nvme*/queue/scheduler`.
- **Read-ahead**: default ~ 128 KB; streaming large files? raise to **a few MB** (`blockdev --setra`) — fewer syscalls, pipelined reads.
- Fastest interface + **enough PCIe lanes**; stripe SSDs with **RAID 0** if one disk can't saturate your GPUs.

*Plain English: the kernel already reads ahead for you — tell it **how far**; and if the whole catalog fits in the front room, move it there once.*

The RAM shortcut

The **page cache** auto-caches recent reads — warm caches speed up moderately sized datasets (petabyte-scale will thrash).

If the dataset **fits in RAM** (incl. CPU+GPU unified memory on Grace Blackwell), **preload it at startup**: an ultrafast in-memory cache, near-zero disk I/O during training.

Part II — GDS: Cut the CPU Bounce Buffer Out of the Path

Disks are tuned. Next question: why does every byte still detour through CPU memory?

- Normal read path: SSD → **CPU memory** → CUDA copy → GPU memory — an extra hop through a **host bounce buffer**.
- **GDS** (GPUDirect Storage): the GPU initiates **DMA** against the SSD or NIC; data lands **directly in HBM**. Works for **local NVMe** and **NVMe-oF**.
- **The CPU still configures and orchestrates** every I/O — what disappears is the copy through host memory, not the CPU's control role.

*Plain English: the parts truck now unloads **straight at the workstation** instead of via the front office — but the office still signs the paperwork.*

Two GPUDirect siblings (book note)

GDS accelerates **storage→GPU** DMA;
GPUDirect RDMA accelerates **network→GPU** DMA.

Both remove the **host bounce buffer**;
neither removes CPU orchestration.

GDS in the Wild: VAST's Before/After Architecture

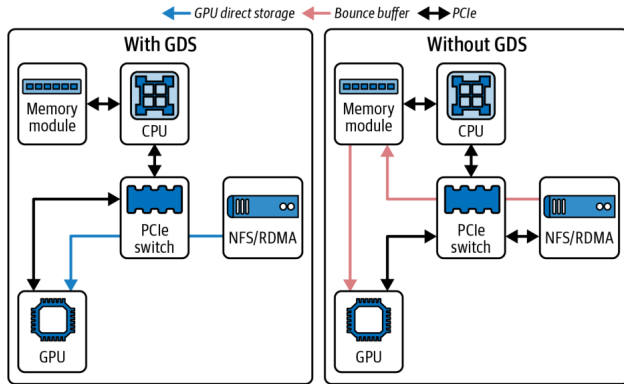


Figure 5-1. VAST Data's network architecture with GDS (left) vs. without (right)

*Plain English: how much the shortcut saves depends on how congested the front office was — measure **your** building, not the vendor's.*

- Right: traditional **staged DMA** — the pink path copies through host memory.
- Left: **direct GPU pull** over the PCIe switch — host memory copies gone, CPU utilization down.
- VAST reports: **+20%** read throughput on an A100; **+30%+** on an H100 — higher NIC bandwidth = greater CPU burden removed.
- Book caveat: **validate on your workload** — uplift varies with I/O size, queue depth, NIC generation, filesystem.

Using GDS in Practice: the cuFile API

- Applications read through CUDA's **cuFile** library: **cuFileRead** works on regular **POSIX file descriptors**; automatic buffer alignment; the **nvidia-fs** kernel driver orchestrates the DMA.
- Async variants **cuFileReadAsync** / **cuFileWriteAsync** put storage I/O **on CUDA streams** (Chapter 11) — overlap and pipelining.
- **Pinned host memory is not used** in the storage→GPU path; cuFile registers **GPU device buffers** instead.

Plain English: one library call, and the forklift route is booked end to end — as long as the loading dock (filesystem) speaks the same protocol.

Requirements and supported stacks

Modern NVIDIA GPU + drivers + CUDA toolkit; DMA-capable storage stack: local NVMe / NVMe-oF on **XFS/EXT4 with O_DIRECT**, **NFS over RDMA**, and parallel filesystems integrating nvidia-fs (**BeeGFS**, **WekaFS**, **VAST**, **IBM Storage Scale**).

- Book tip: use **O_DIRECT** when possible; cuFile tolerates non-O_DIRECT descriptors, but **misalignment can cost extra copies**.

When Does GDS Actually Help?

The honest answer (book)

If the CPU was **easily handling** the transfers, GDS may not change throughput much — but it still **lowers CPU usage**, freeing cores for preprocessing. If the CPU is **saturated with memcpys**, GDS **helps a lot**.

- Worked example: $1,000 \text{ batches/s} \times 1 \text{ MB} = 1 \text{ GB/s}$. Copying that through the CPU eats **a few cores**; with GDS the GPU pulls it directly — at higher rates or thousands of GPUs, the effect compounds.

Plain English: the shortcut pays most when the office clerk was the queue — and even when it isn't faster, the clerk is now free to do other work.

- Training is **overwhelmingly read-heavy** — most GDS gains are measured on reads.
- Fast **checkpoint writes** matter too: RDMA-accelerated **writes** require filesystem support — e.g., **WekaFS** ships GDS-aware plugins for both read and write paths.
- Vendor ecosystem (WekaIO, DDN, VAST, Clouddian...) means GDS often works **out of the box** on enterprise NAS / parallel filesystems.

Measuring It: gdsio Before and After

```
# Before: storage -> CPU memory (CPU path, -x 2)
$ /usr/local/cuda/gds/tools/gdsio \
  -f /mnt/data/large_file \
  -d 0 -w 4 -s 10G -i 1M -I 0 -x 2
Total Throughput: 8.0 GB/s Avg Latency: 1.25 ms
```

```
# After: storage -> GPU memory via GDS (-x 0)
$ /usr/local/cuda/gds/tools/gdsio \
  -f /mnt/data/large_file \
  -d 0 -w 4 -s 10G -i 1M -I 0 -x 0
Total Throughput: 9.6 GB/s Avg Latency: 1.00 ms
```

Same file, same workers/size/IO-size; only the **transfer selector** (-x) changes — compare paths consistently.

Plain English: one tool, two flags, a clean 20% verdict — run it before you argue about architecture.

Path	Thpt	Latency
Storage → CPU	8.0 GB/s	1.25 ms
Storage → GPU	9.6 GB/s (+20%)	1.00 ms (−20%)

Table 5-1. **gdsio** ships under `/usr/local/cuda/gds/tools` — benchmark **your** disk-to-GPU path before and after enabling GDS, while also freeing the CPU cycles previously spent in host buffers.

Checkpointing GPU State with cuda-checkpoint *

* A sidebar in the storage story: snapshotting a whole CUDA process, not writing model checkpoints.

- **cuda-checkpoint** + a CPU checkpointer (**CRIU**) snapshot a **running CUDA process**: lock CUDA entry points, **drain outstanding work**, copy device memory **to host**, release the GPU.
- Driver API: `cuCheckpointProcess{Lock, Checkpoint, Restore, Unlock}`. Restore needs persistence mode; can **remap to different GPUs** of the same chip type.
- Suspend time $\approx$ **device-memory image size** $\div$ **host-link bandwidth** — verify with Nsight Systems markers.

Plain English: it freezes the whole workshop mid-shift so it can reopen elsewhere — different from photocopying the blueprints (model checkpoints).

Two clarifications (book notes)

- **Not** a GDS path: device memory goes **GPU $\rightarrow$ host** first; CRIU then persists the process image.
- **Orthogonal** to framework checkpoints (PyTorch state-dicts) — use it to **complement, not replace** them: fault tolerance, preemption, migration of long-running jobs.

DeepSeek's Fire-Flyer File System (3FS): Codesigning Storage for AI

- Born from one observation: AI does **massive random reads** — read caching is **counterproductive**. 3FS: **no cache, direct I/O**, straight to NVMe.
- Four components — cluster manager, metadata service, storage service, client — over an **RDMA fabric** (InfiniBand/RoCE).
- Reported: **6.6 TB/s** aggregate reads on 68 nodes (+1.4 TB/s background); up to **7.3 TB/s** — vs. Ceph's ~1.1 TB/s on similar hardware.
- Book caveat: building your own FS is a **big investment** — most teams should start from existing systems (next part).

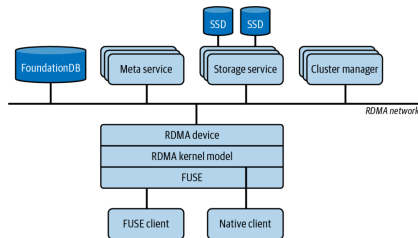


Figure 5-2. Components of DeepSeek's Fire-Flyer File System (3FS)

FUSE caveat (book warning)

A **FUSE** (user-space) filesystem **cannot** deliver a GDS path — GDS needs **kernel-level** `O_DIRECT` integration (NVMe/NVMe-oF, BeeGFS, WekaFS, IBM Storage Scale, VAST).

Plain English: DeepSeek rebuilt the warehouse around how AI shops: random grabs, no shelves, forklifts only.

Part III — NFS: Simple to Set Up, Simple to Saturate

Custom filesystems are the exception. What do the rest of us mount?

- One shared NFS server = every node reads the same place — convenient, and a **throughput bottleneck** once many nodes read at once.
- If you must use NFS: **multiple fast NICs**, or **multiple servers** each holding a partition of the dataset; NVMe (even RAID 0) underneath.
- Book verdict: NFS **only for modest scale** (a few nodes) — larger clusters want parallel filesystems or cloud caches (next page).

Client-side tuning that actually matters

rsize/wsize at the max (1 MB); kill access-time writes; cache attributes:

```
mount -o rsize=1048576,wsiz=1048576,\
noatime,async,actimeo=60,lookupcache=pos \
nfs-server:/data /mnt/data
```

1 MiB requests + noatime + 60 s
attribute/directory caching — simple tweaks that
vastly cut per-request overhead on large shared
datasets.

Plain English: one shop counter serving the whole city works fine — until the whole city shows up at once.

Parallel Filesystems and Object Stores

Parallel FS: stripe for bandwidth

Lustre serves data from multiple **OSTs** (object storage targets). **Stripe** large files across OSTs (`lfs setstripe`, e.g. 4–8): $4 \text{ OSTs} \times 500 \text{ MB/s} = \mathbf{2 \text{ GB/s}}$ theoretical read peak per file.

- Monitor during training (**lmt**, vendor dashboards): a few **hot** storage nodes usually means a **sharding imbalance** — find out why.

Plain English: ten loading docks beat one; and if the goods live overseas (S3), either ship ahead of time or rent a local depot.

Object stores: stage or cache

Naive S3 reads during training are slow. Either **stage to local NVMe** before training (**s5cmd**, AWS C++ SDK — highly parallel tools), or put a **cache layer** on top (Amazon FSx for Lustre over S3).

- Streaming directly? Use **large range-GETs**, multithreaded.
- Book tip: **verify** the cache's performance uplift actually justifies its cost — work with the provider if numbers disappoint.

Two Blunt but Effective Levers: Replicate and Compress

Replicate the dataset to every node

Brute force: copy the **whole dataset** onto each node's disk — **zero network reads**, immediate win, at the cost of storage. Common and very effective when node storage allows.

Store compressed, decompress on the fly

JPEGs, compressed Arrow/Parquet: trade **I/O bandwidth** for CPU/GPU cycles — reasonable exactly when **I/O is the bottleneck** and compute is idle.

- Decode on the GPU: **nvJPEG** for images; Blackwell's on-die **Decompression Engine** handles **LZ4, Snappy, Deflate** in-pipeline — SMs stay free for compute kernels. **Favor these formats** for I/O-bound work.
- **NVLink-C2C** (~900 GB/s bidirectional) keeps CPU-assisted stages from becoming the new bottleneck.
- The balance test: if **decompression time replaces I/O** as the bottleneck, the compression wasn't worth it.

Plain English: either give every workshop its own copy of the catalog, or ship the catalog vacuum-packed — just make sure unpacking isn't slower than shipping was.

Monitoring Storage I/O: the Toolbox

Host OS tools

iostat, **iotop**, **nvme-cli**, **perf**, **eBPF** + vendor dashboards — queues, latencies, read-ahead effects, cache hit ratios; are local NVMe or the network links to NAS/object storage **saturated**?

- **DCGM** (Data Center GPU Manager) adds GPU-side I/O statistics — together they expose **GPU starvation due to I/O**.

Plain English: same rule as every chapter: no verdict without a measurement — the disk, the NIC, and the GPU each get their own gauge.

GDS-aware tracing

Nsight Systems with `--trace=gds` captures **cuFile API activity** on the timeline; enable cuFile static tracepoints via `/etc/cufile.json`.

- Caveat: kernel-mode counters for NVMe **peer-to-peer DMA** aren't exposed in Nsight Systems and may be missing on some GDS stacks.

Whose Fault Is the Idle GPU? A Two-Timer Decomposition

- In PyTorch, timing `next(data_iterator)` measures **total GPU idle time waiting for a batch** — prefetching + H2D copy included, not just Python.
- Split it in two: **(1) Python cost** — set `num_workers=0` and time the bare iterator pull; **(2) H2D copy cost** — wrap `.to("cuda")` in **CUDA events**, or read the **Copy lanes** in Nsight Systems.
- Compare both against total idle time $\Rightarrow$ speed up the **Python pipeline** (workers, simpler transforms) or the **transfer path** (pinned memory, interconnect, GDS).

*Plain English: first clock **how long** the crew stands idle; then split the blame between the kitchen (Python) and the delivery van (H2D copy).*

Why this matters (book example)

GPUs waiting **30%** of the time for data $\Rightarrow$ throughput capped by I/O. After tuning: waiting **5%** — training steps/s up **proportionally**.

- Storage tuning = killing many small inefficiencies (5 ms here, a small buffer there) — at scale they **add up**.

Part IV — The Data Pipeline: The Last Mile to the GPU

Storage delivers bytes; someone still has to turn bytes into batches.

- Typical loading path: **read** from storage → **decode/deserialize** (JSON, JPEG) → **transform** (tokenize, crop) → **collate** into batches.
- These steps are CPU-intensive — but compute-heavy ones can be **offloaded to the GPU**.
- Goal: GPUs **never idle** waiting for data, with the **right amount** of CPU work running in parallel.

Perspective (book)

A poorly tuned input pipeline can waste **50%** of GPU time; algorithmic tweaks often buy only **a few percent**. Optimizing the pipeline frequently **beats model-level cleverness**.

Plain English: the last mile decides the delivery date — a warehouse full of parts feeds nothing if the kitting bench is slow.

DataLoader Essentials: the Five Knobs

Knob	What it does	Guidance
<code>num_workers</code>	parallel fetch+preprocess processes (GIL-free)	empirical; watch CPU + disk
<code>pin_memory=True</code>	page-locked host buffers	enables real async H2D
<code>non_blocking=True</code>	async <code>.to(device)</code> copies	pairs with pinned memory
<code>persistent_workers=True</code>	workers survive across epochs	kills respawn overhead
<code>prefetch_factor</code>	batches queued per worker (default 2)	4–8 for bursty I/O

- Too few workers $\Rightarrow$ idle GPU; too many $\Rightarrow$ contention for CPU cores and I/O bandwidth. Aim for **$\sim 100\%$ disk throughput, headroom on CPU**.
- High-core CPUs (72-core **Grace**) can host more workers — mind diminishing returns.
- Scaling out GPUs? **Scale the pipeline too**: each rank reads its own shard; aggregate loading work **grows with cluster size** — or the bottleneck shifts to ingestion.
- Set a high `ulimit -l` (memlock) so large pinned buffers can allocate.

Plain English: five dials on one machine — and more chefs (GPUs) always means more prep cooks (workers).

The Overlap in Code: Pinned Memory + Prefetch + Streams

```
loader = DataLoader(
    dataset, batch_size=B,
    num_workers=8, pin_memory=True,
    persistent_workers=True, prefetch_factor=4)

copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()
for batch in loader:
    with torch.cuda.stream(copy_stream):
        batch_gpu = batch.to(device, non_blocking=True)
        # H2D must finish before compute uses it
    with torch.cuda.stream(compute_stream):
        torch.cuda.current_stream().wait_stream(copy_stream)
        outputs = model(batch_gpu)
```

- 8 workers, each keeping **4 batches** queued in **pinned** memory ($\text{num_workers} \times \text{prefetch_factor}$ total).
- Pinned source $\Rightarrow$ non_blocking copy uses **DMA**: while the GPU computes batch N , batch $N+1$ is **already transferring**.
- Without pinning, memory is pinned **on the fly per transfer** — latency you didn't order.

Plain English: the prep station always keeps the next tray loaded — the chef never looks up between dishes.

Avoid Python Bottlenecks; Profile the Loader in Isolation

- Red flag: **pure-Python loops** tokenizing lines one by one. Vectorize, or use **Rust/C++-backed** libraries (Hugging Face Tokenizers, TorchText) — Python is just the interface.
- Prefer **batched transforms**: collate first (custom `collate_fn`), then transform **whole tensors** — some ops must stay per-sample; know your workload.
- Hidden bottlenecks sneak in easily: **debug logging**, expensive CPU transforms — often visible **only under load**.

Plain English: time the prep kitchen with the dining room closed — just remember service adds its own noise when you reopen.

Baseline the loader alone

Time how long the DataLoader takes to produce **100 batches** with all GPU work disabled; compare with target iteration time and measured GPU-idle time. Loader too slow? Fewer logs, simpler transforms, **more workers**.

- Caveat (book note): disabling GPU work also removes **kernel-launch overhead**, so isolated loader throughput reads **lower** than in a real run — still useful, just remember it.

Multimodal Preprocessing on the GPU: NVIDIA DALI

- **DALI** moves decoding/augmentation to the GPU (or optimized C++ CPU code): JPEG decode, random crop, resize, normalize — using the GPU's **media-acceleration hardware**.
- Declarative **static graph**: subclass Pipeline, declare ops in `define_graph()`; DALI handles execution, prefetching, threading.
- Book example: CPU at **800%** (8 cores) with a stalling GPU → with DALI, CPU at **200%** while the GPU decodes **concurrently with compute**.

Plain English: moving the chopping board next to the stove helps only if the food doesn't travel back to the pantry between steps.

The placement pitfall

Using DALI **only to decode**, then handing pixels **back to the CPU** for augmentation adds **host-device-host** copies that can **negate the gains**.

- Alternative (book tip): **fuse** GPU-friendly preprocessing into your GPU graph with TorchVision / TensorRT / custom CUDA kernels — may beat DALI end to end.
- Benchmark **three ways**: CPU-only vs. DALI vs. fully fused GPU graph.

Prepare Data Offline: NVIDIA NeMo Curator

- **NeMo Curator** (open source) prepares **multiterabyte LLM datasets** ahead of training: cleansing, tokenizing, shuffling, **deduplication**, quality filtering — distributed across **multiple GPUs/nodes**.
- Packs data into **few large files**; NeMo training reads memory-mappable **.bin + .idx**; Curator I/O is sharded JSONL/Parquet.
- Can also generate **synthetic data** to augment increasingly scarce human data.

Why offline prep wins

Training becomes **mostly forward/backward passes**: consistent, evenly sized, padded inputs; **no on-the-fly tokenization**, no millions of small files.

- Book tip: store **N differently shuffled copies** to avoid runtime shuffling across N epochs — disk space for speed.
- “**Almost never** train with raw text.”
NeMo loading still runs on **CPUs** — combine with **GDS** to bypass CPU I/O.

Plain English: do the mise en place the night before — during service you only plate.

Part V — The Continuous Profiling and Tuning Loop

Everything so far is a one-time fix. Performance, though, is a feature that needs regression testing.

- ❶ **Establish a baseline:** single GPU → one node → multinode, tracking samples/s. N GPUs should give $\sim N\times$; 8 GPUs at $5\times =$ **62.5% efficiency** — quantify the gap.
- ❷ **Profile the multi-GPU run (nsys):** GPUs idle **during all-reduce** $\Rightarrow$ communication; idle **at iteration start** $\Rightarrow$ data loading; check CPU timelines and sync points too.
- ❸ **Zoom into kernels (ncu)** when one op underperforms — book example: an NCCL kernel at **60% SM utilization** over PCIe; on NVLink, **90%**.

Plain English: treat performance like a test suite: it either passes tonight, or you find out tomorrow which commit broke it.

The loop

Run → Measure → Tune → Run → Measure
→ Tune → ...

- Every kernel verdict lands in one of three bins: **network-bandwidth, memory-bandwidth, or compute bound**.
- It's much easier to **maintain** good performance than to **regain** it later.

Identify the Cause, Fix One Thing, Remeasure

Map bottleneck → hypotheses → fixes

- **Network**: GPUDirect RDMA; bigger messages; NCCL_SOCKETS_PER_THREAD; NUMA-aware binding.
- **Straggler GPU**: data imbalance or extra logging on one rank — move it off the critical path.
- **CPU/data**: more workers, DALI offload, more offline preprocessing.
- **Sync**: remove stray `torch.cuda.synchronize()` / barriers (Ch. 13).

- **Change 1–2 things at a time**, then remeasure — otherwise you can't credit the right fix.
- **Updates shift optima**: new NCCL/CUDA/PyTorch can help — rerun the workflow after every upgrade.
- **Automate**: nightly profiling runs, dashboards, alerts on utilization drops.
- **Document** tacit knowledge in code/config —
“NCCL_SOCKET_NTHREADS=2 gave +10% on this cluster.”

Plain English: tune like a scientist: one variable, one measurement, one written-down conclusion.

Communication- or Compute-Bound? The Batch-Size Experiment

- Setup: gradient all-reduce uses **60 GB/s of a 100 GB/s NIC**. Clogged network, or slow GPUs?
- Key fact: all-reduce traffic scales with **parameter count, not batch size** — batch size varies **compute** at **fixed communication**.
- **Halve the batch**: NIC stays at 60 GB/s $\Rightarrow$ **network-bound**; drops to 40 GB/s $\Rightarrow$ **compute-bound** (GPUs starve the NIC).
- **Double the batch** to confirm: NIC pinned at 60 $\Rightarrow$ communication limits; **all-reduce share of time shrinks** $\Rightarrow$ compute limits.

Plain English: hold the shipping volume constant and vary the workload — whichever side refuses to move is your bottleneck.

Read it like a roofline

Plot **achieved GB/s** and **% time in communication** vs. batch size — the plot shows where the ceiling is and **which subsystem to tune**.

- Nsight tip: long **NCCL-wait** gaps between kernels $\Rightarrow$ communication-bound; busy GPUs below expected FLOPS $\Rightarrow$ memory- or compute-bound (ncu / PyTorch profiler).

Key Takeaways (the Book's Three)

1 — Scale the input pipeline with compute

More GPUs demand more storage bandwidth **and** more loader parallelism — or added GPUs give **zero speedup**.

2 — Use the right tool for the job

NCCL for collectives (training); **NIXL** for point-to-point token streaming (inference); **GPUDirect RDMA / GDS** remove the bounce buffer for network / storage; always **DistributedDataParallel** over DataParallel.

Plain English: feed the factory, take the shortcut past the front office, and never argue about the bottleneck — measure it.

3 — Profile end to end

The bottleneck is **not obvious**: GPUs are compute-, communication-, or I/O-bound. Nsight Systems + Nsight Compute + PyTorch profiler + NCCL logs tell you **which** — then this chapter tells you what to do about it.

- These libraries are **purpose-built and heavily tuned** — leverage them instead of reinventing the wheel.

Conclusion and What's Next

This chapter in one sentence

Fast data movement is as critical as raw compute — the fastest GPU in the world provides little benefit if it's constantly waiting for data from storage.

- Storage is a **foundational layer**: NVMe + GDS + tuned pipelines shorten training time and **raise the rate of experimentation**.
- You **don't need custom I/O solutions**: NVIDIA and the open source community ship highly tuned, purpose-built tools — focus on your model, data, and application.
- Full-stack thinking continues: overlap communication/computation, use the fastest links available, keep every pipe full — **faster time-to-insight, better utilization, lower cost**.
- Next in the book: **Chapter 6** — GPU architecture, CUDA, and occupancy (covered in our previous session); the storage principles here stay relevant at **every** layer above.

Questions?

References & Further Reading

- ❶ C. Fregly. *AI Systems Performance Engineering* (O'Reilly, 2026), Chapter 5.
- ❷ NVIDIA. *GPUDirect Storage Documentation* — cuFile API reference, gdsio benchmarking guide, `cufile.json` configuration.
- ❸ NVIDIA. *cuda-checkpoint* (GitHub) and CRIU — Checkpoint/Restore in Userspace.
- ❹ DeepSeek. *Fire-Flyer File System (3FS)* — open source repository and Fire-Flyer AI-HPC paper (arXiv:2408.14158).
- ❺ NVIDIA. *DALI Documentation* — pipelines, GPU decode, framework integration.
- ❻ NVIDIA. *NeMo Curator* — documentation and GitHub repository.
- ❼ Linux kernel documentation — `blk-mq`, I/O schedulers, `io_uring`; PyTorch docs — `DataLoader`, `DistributedSampler`.
- ❽ VAST Data. *GPUDirect Storage benchmark reports* (A100/H100 read-throughput uplifts).